

Individual Assignment : Queue Simulation

Ndeye Anta MBAYE

September 19, 2026

1 Python Code for Queue Simulation

```
1 # Author: Ndeye Anta Mbaye
2 # Date: 19 September 2026
3 # simulation.py
4
5 # Imports
6 import numpy as np
7 import pandas as pd
8 import matplotlib.pyplot as plt
9
10 # Simulate 1,000 scenarios.
11 scenarios = []
12
13 # For each scenario:
14 for i in range(1000):
15     # Generate the arrival rate, lambda, randomly between 8 and 12
16     # customers/hour.
17     arrival_rate = np.random.uniform(low=8, high=12)
18     # Generate the current service rate, mu, randomly between 13
19     # and 17 customers/hour.
20     service_rate = np.random.uniform(low=13, high=17)
21
22     # For each scenario, calculate rho, Ls, Lq, Ws, Wq
23     rho = arrival_rate / service_rate
24     ls = rho / (1-rho)
25     lq = ls-rho
26     ws=ls/arrival_rate
27     wq=lq/arrival_rate
28
29     # Append the scenarios to the array
30     scenarios.append({
31         'arrival_rate': arrival_rate,
32         'service_rate': service_rate,
33         'rho': rho,
34         'ls': ls,
35         'lq': lq,
36         'ws': ws,
```

```

35         'wq': wq
36     })
37
38     # Create histograms showing the distributions of rho, Ls, Lq, Ws,
and Wq
39     keys = ["rho", "ls", "lq", "ws", "wq"]
40     for key in keys:
41         values = [scenario[key] for scenario in scenarios]
42         plt.hist(values, bins=30)
43         plt.title(f"Distribution of {key} with 30 bins")
44         plt.xlabel(key)
45         plt.ylabel("Frequency")
46         plt.show()
47
48     # Save the simulation results as a CSV file
49     pd.DataFrame(scenarios).to_csv("results/scenarios.csv", index=
        False)
50
51     # Find the descriptive stats
52     stats = []
53     labels = ["arrival_rate", "service_rate", "rho", "ls", "lq", "ws",
        "wq"]
54
55     for label in labels:
56         values = [scenario[label] for scenario in scenarios]
57
58         # Find the stats for each metric and append to array
59         stats.append({
60             "variable": label,
61             "mean": round(np.mean(values),3),
62             "median": round(np.median(values),3),
63             "mode": pd.Series(values).mode().iloc[0],
64             "standard deviation": np.std(values),
65             "min": np.min(values),
66             "max": np.max(values)
67         })
68
69     # Save descriptive statistics as csv
70     pd.DataFrame(stats).round(3).to_csv("results/
        descriptive_statistics.csv", index=False)
71
72
73     # Assume management wants at least 95% of customers to wait no
more than 15 minutes.
74     # Use the M/M/1 relationship,  $P(W_q > t) = \rho * \exp[-(\mu - \lambda)t]$ 
where  $t = 15/60$  hours.
75     results = []
76     percentiles = [50, 75, 90, 95, 97.5]
77
78     # Use the 97.5th percentile of lambda as the design demand level.

```

```

79 # Repeat the capacity calculation using the 50th, 75th, 90th, 95th
    , and 97.5th percentiles of demand.
80 for percentile in percentiles:
81     lambda_p = np.percentile([scenario["arrival_rate"] for
    scenario in scenarios], percentile)
82     # Determine the minimum service rate mu required to achieve
    the 95% target.
83     min_rate = lambda_p
84
85     while (lambda_p / min_rate) * np.exp(-(min_rate - lambda_p) *
    (15 / 60)) > 0.05:
86         min_rate += 0.01
87
88     rho = lambda_p / min_rate
89     p= rho * np.exp(-(min_rate - lambda_p) * (15 / 60))
90     results.append({
91         "lambda": lambda_p,
92         "min_rate": min_rate,
93         "rho": rho,
94         "probability": p,
95     })
96
97 # Sensitivity table
98 sensitivity_table = pd.DataFrame(results, index=[f"{p}th
    percentile" for p in percentiles], columns=["lambda", "min_rate
    ", "rho", "probability"])
99 print(sensitivity_table)

```

2 Histograms

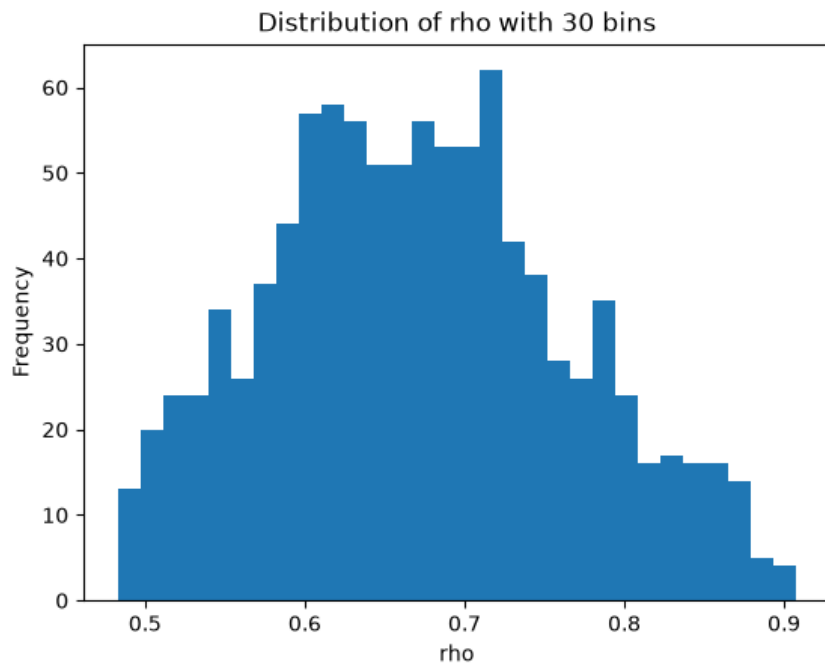


Figure 1: Distribution of ρ

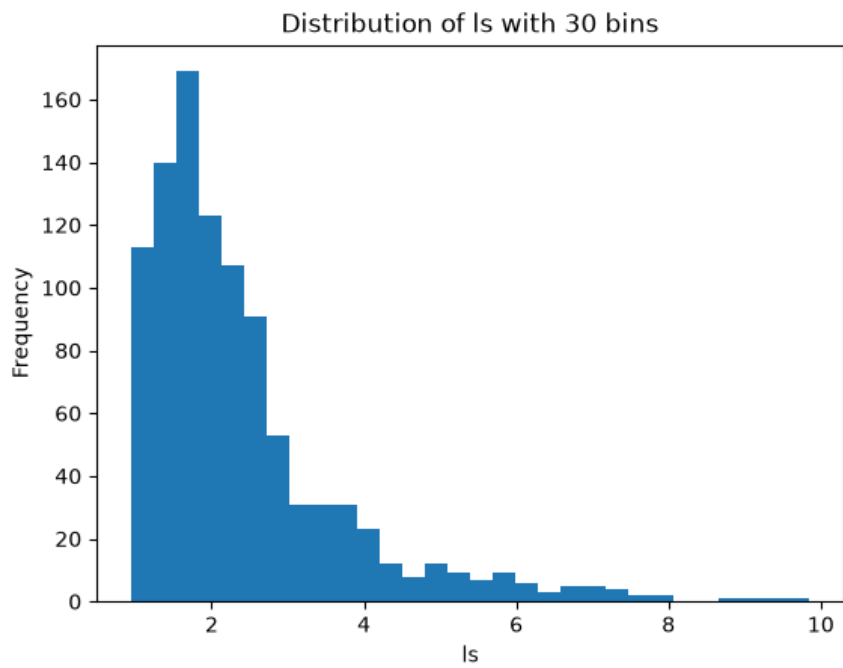


Figure 2: Distribution of ls

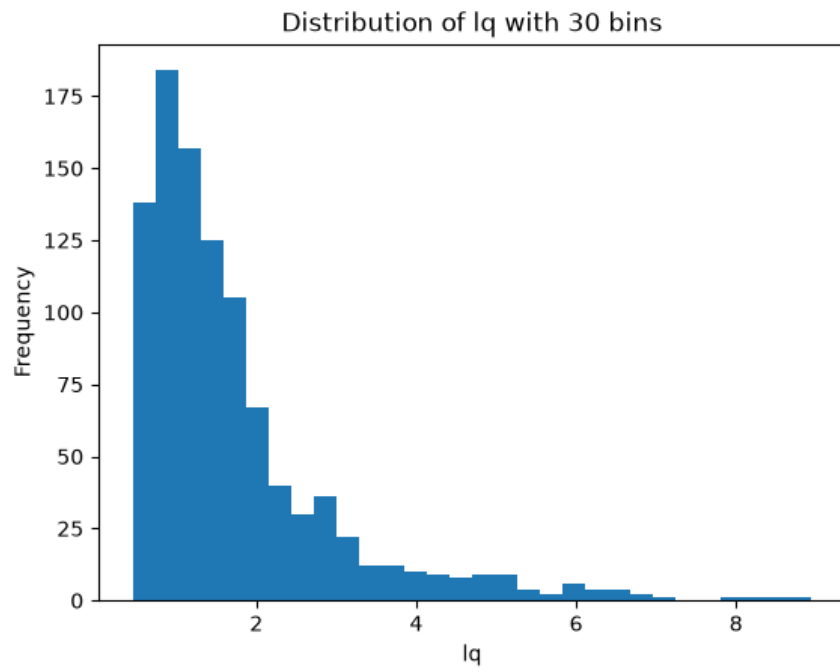


Figure 3: Distribution of L_q

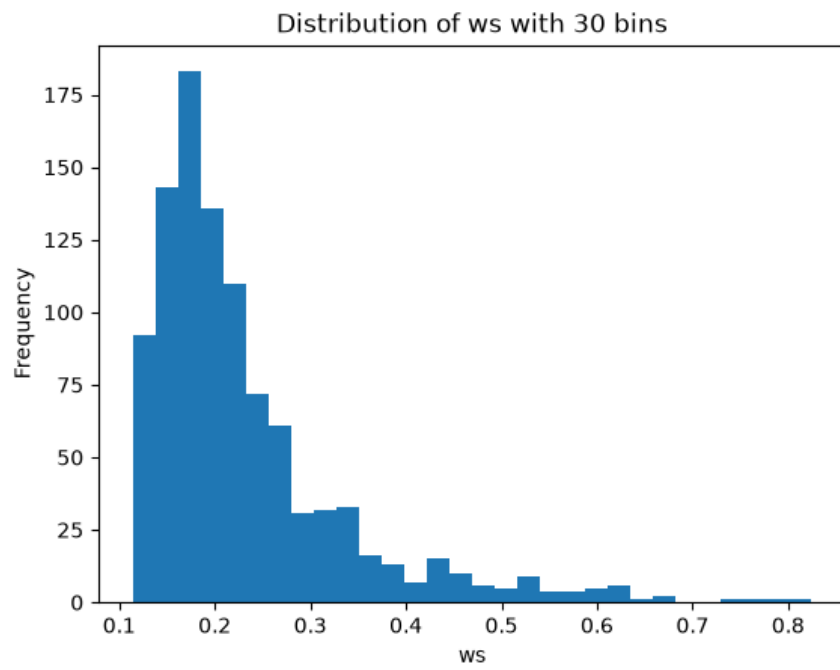


Figure 4: Distributions of W_s

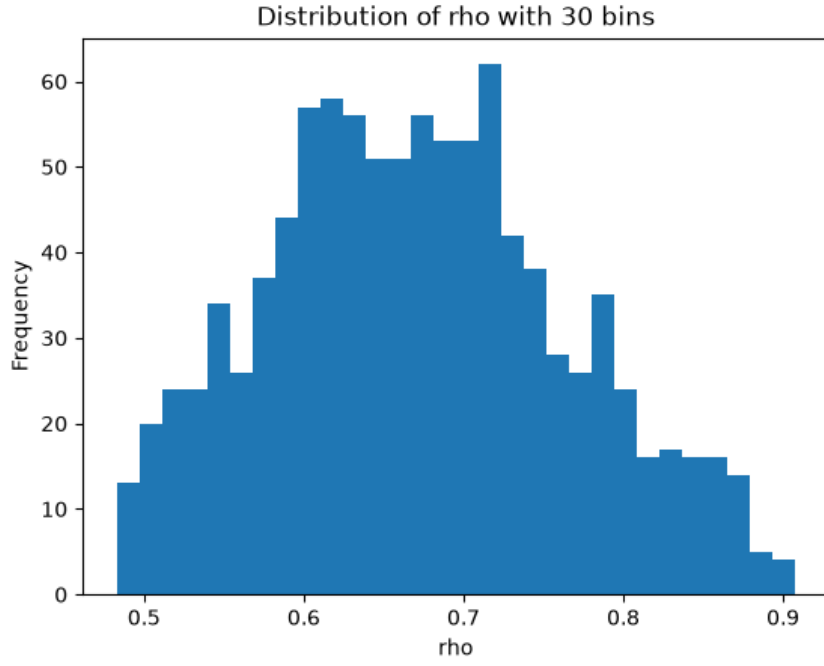


Figure 5: Distributions of W_q

3 Sensitivity Analysis Table

	lambda	min_rate	rho	probability
50th percentile	10.043634	19.403634	0.517616	0.049861
75th percentile	10.955838	20.445838	0.535847	0.049966
90th percentile	11.570786	21.150786	0.547062	0.049877
95th percentile	11.780404	21.380404	0.550991	0.049985
97.5th percentile	11.890464	21.510464	0.552776	0.049897

Figure 6: Sensitivity Analysis Table

4 Service Capacity Decision Table

Service Capacity Decision Table		
	Metric	Value
0	Design Demand Level	97.5th percentile
1	Arrival rate	11.890464
2	Minimum service rate	21.510464
3	P($W_q > 15$ minutes)	0.049897
4	Target probability	0.05
5	95% target achieved?	True

Figure 7: Service Capacity Decision Table

5 Results Interpretation

The simulation results show that the arrival and service rates vary within their respective ranges, leading to different values of ρ , L_s , L_q , W_s , and W_q , which affect the performance of the queueing system differently. The histograms illustrate the distributions of these metrics across the 1,000 simulated scenarios.

The arrival rate for the 97.5th percentile is approximately 11.9 customers/hour, and the minimum service rate required to achieve the 95% target probability is approximately 21.5 customers/hour. The sensitivity analysis table shows that the arrival and minimum service rates increase as we move from the 50th to the 97.5th percentile, indicating that higher demand levels require higher service rates to maintain the desired performance level.